

Experiment-6

AIM- To configure SPI and read three-axis acceleration data from the LSM6DSL sensor.

APPARATUS- STM32 Discovery Kit, STM32 CUBE IDE, STM32 CubeMX, Realterm.

PROCEDURE

1. Open CubeMX and Click on ACCESS TO MCU SELECTOR.
2. Write and select the microcontroller as you want to use and then click start the project. Select commercial part number STM32L475VGT6.
3. **Clock Selection:** Go to RCC and configure clock as “By pass Clock Source” as shown below:

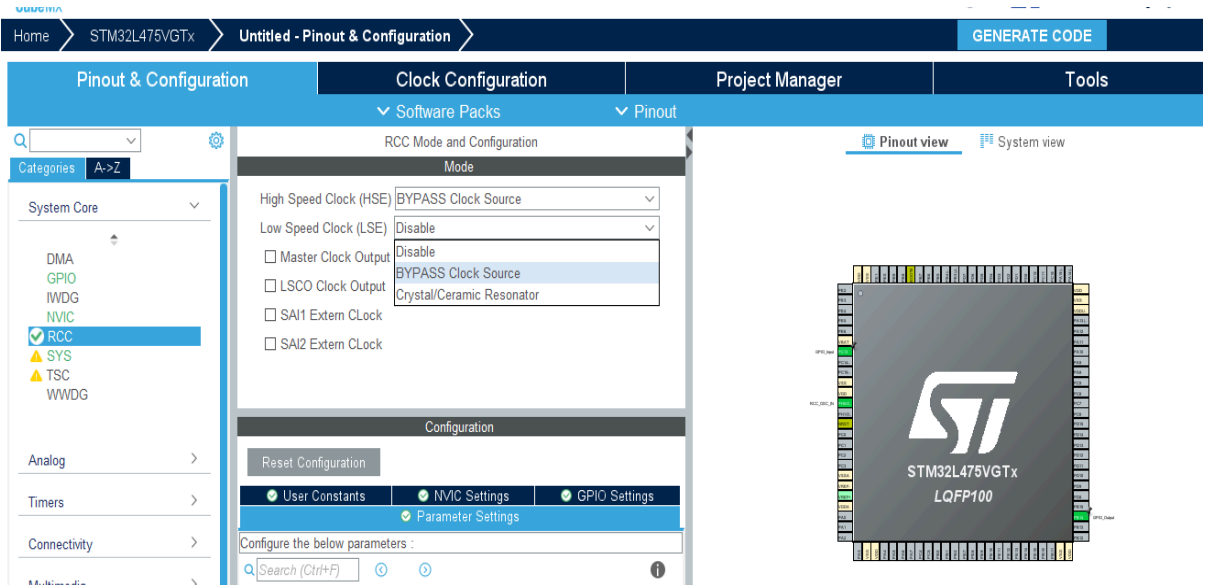


Fig-6.1 – Clock Selection

4. As per user manual of board ST LINK is internally connected to PB6(Tx) and PB7(Rx) of USART1.

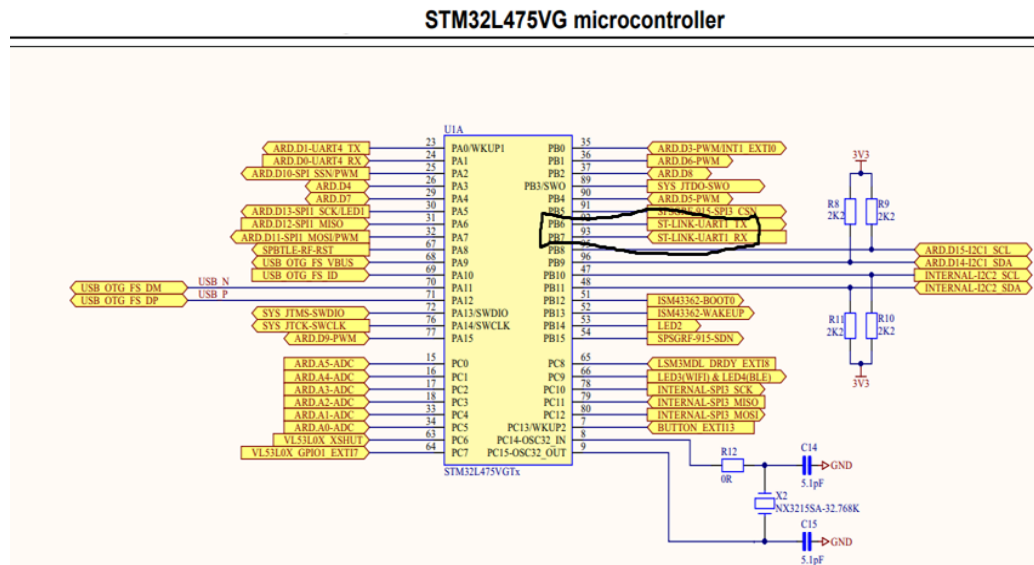


Fig-6.2 – STM32 Board Pin Configuration

- Under “connectivity” click on “USART1” and make PB6 as USART1_Tx and PB7 as USART2_Rx as shown below:

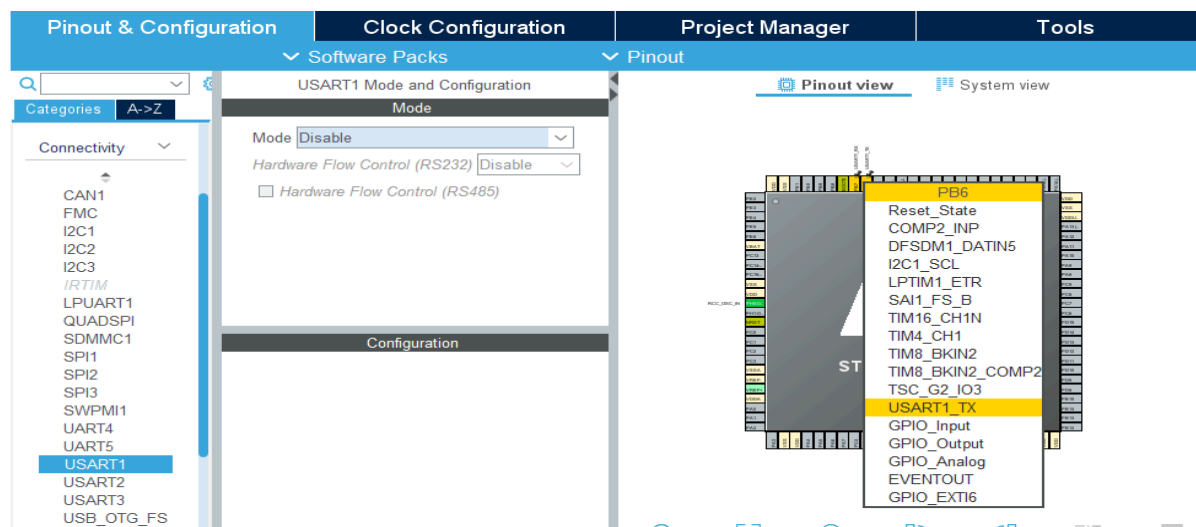


Fig-6.3 – STM32 Board Pinout and Configuration

- Select USART1 and click on “Asynchronous” in “Mode” as shown below :

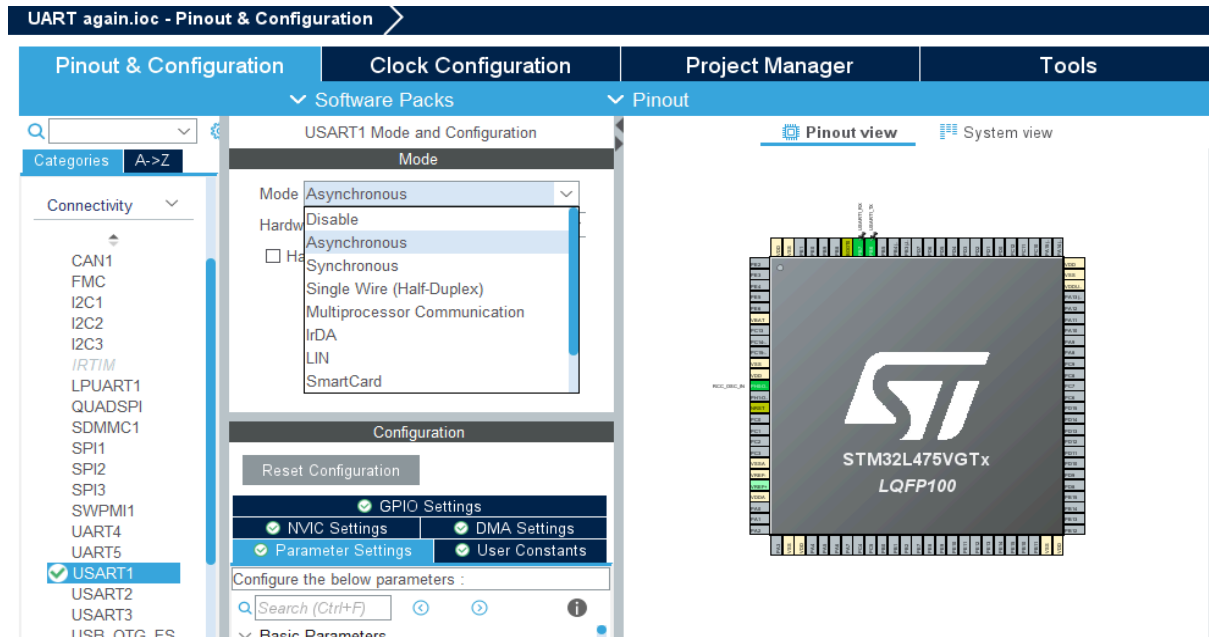


Fig-6.4 – Selection of Asynchronous mode

7. Go to “connectivity” and also select I2C1 in mode.
8. Click on PB8 and select I2C1_SCL to make PB8 pin as clock source pin as shown below:

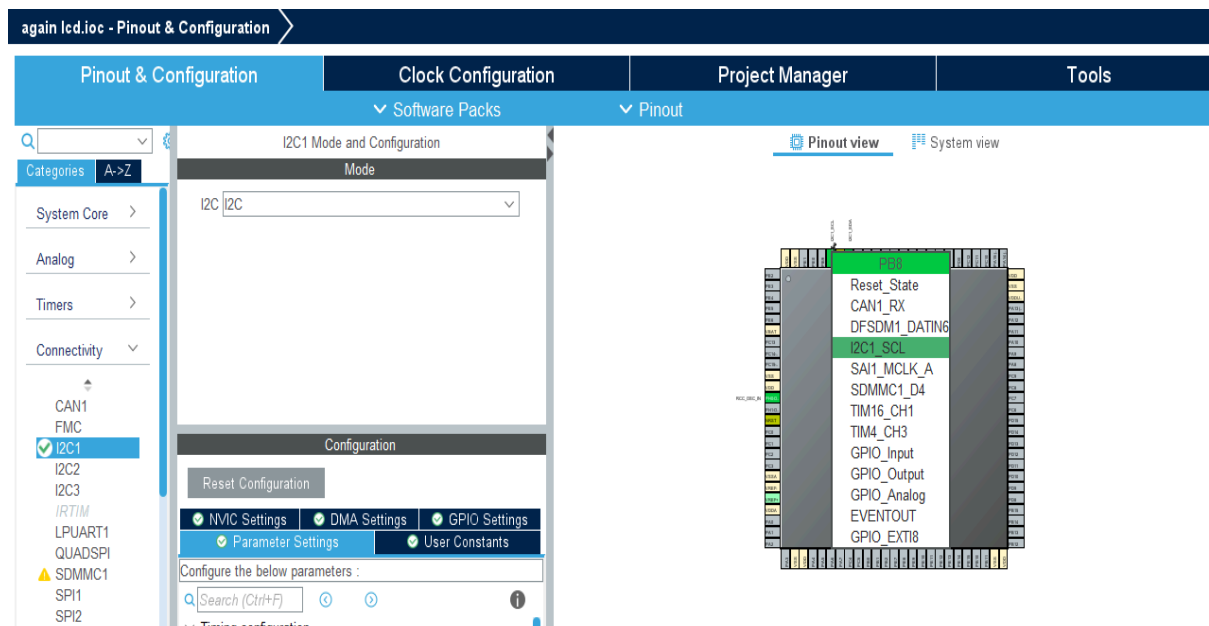


Fig-6.5 – Settings of Basic Parameters

9. **Clock Configuration:** Configure the clock as shown below to get a maximum frequency of 80MHz.

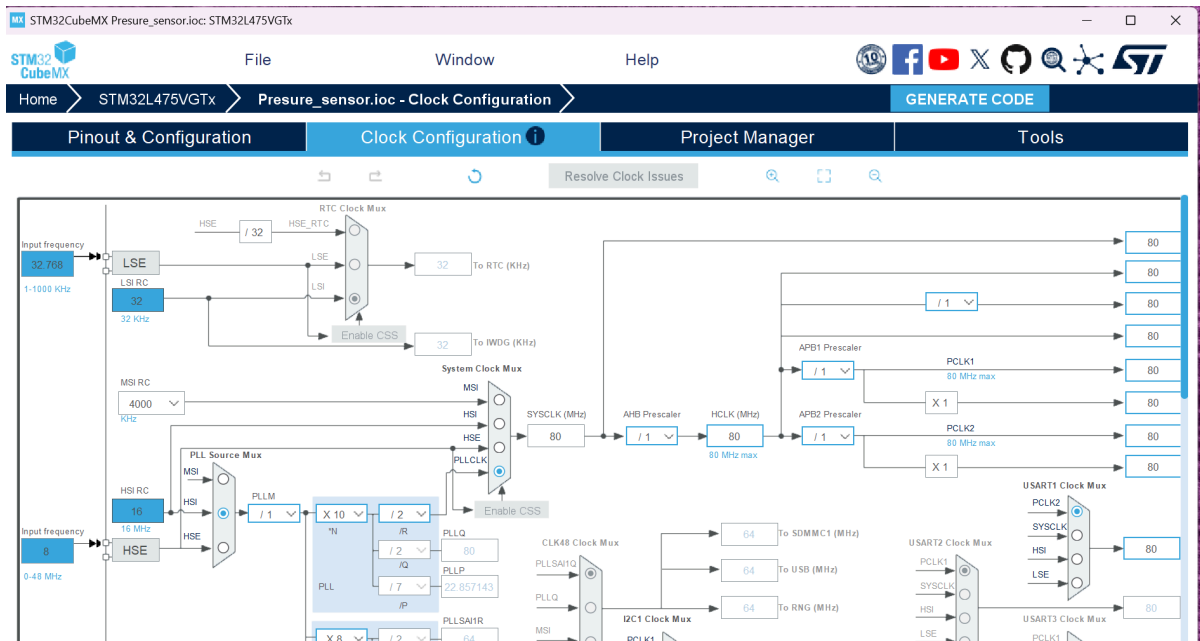


Fig-6.6 – Clock Configuration

10. Write Project name and select CubeIDE.
11. Click on generate code.
12. Click on open project.
13. Click 'OK' on the generated popup showing “successfully imported the project on the work space” and the project will be successfully added to the cube IDE.
14. Copy BSP drivers to your project.
15. The last version of the STM32CubeL4 package is downloaded by default in the STM32CubeMX repository
(C:\Users\user_name\STM32Cube\Repository\STM32Cube_FW_L4_Vx.xx.x\Drivers).

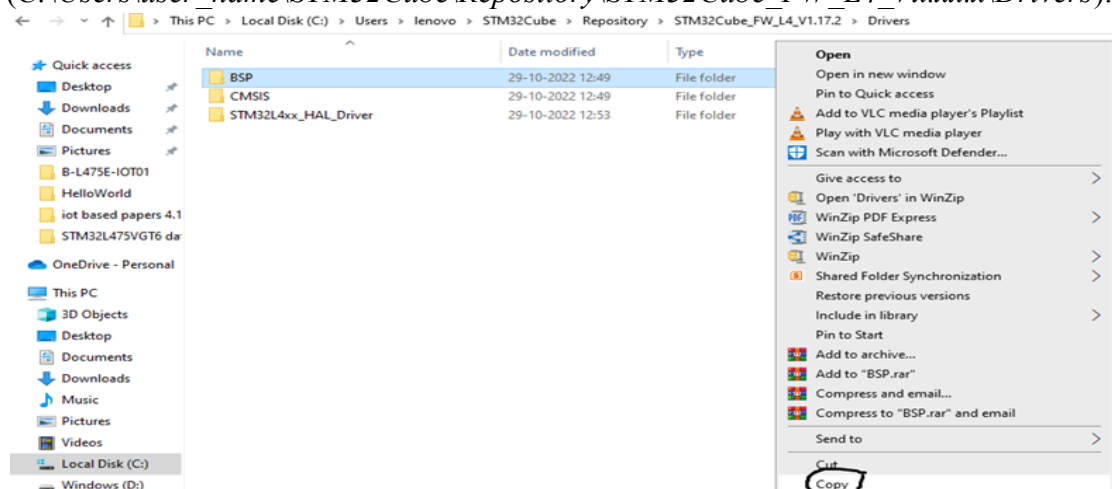


Fig-6.7 – BSP drivers location

16. Paste the copied BSP drivers in the drive folder as shown below:

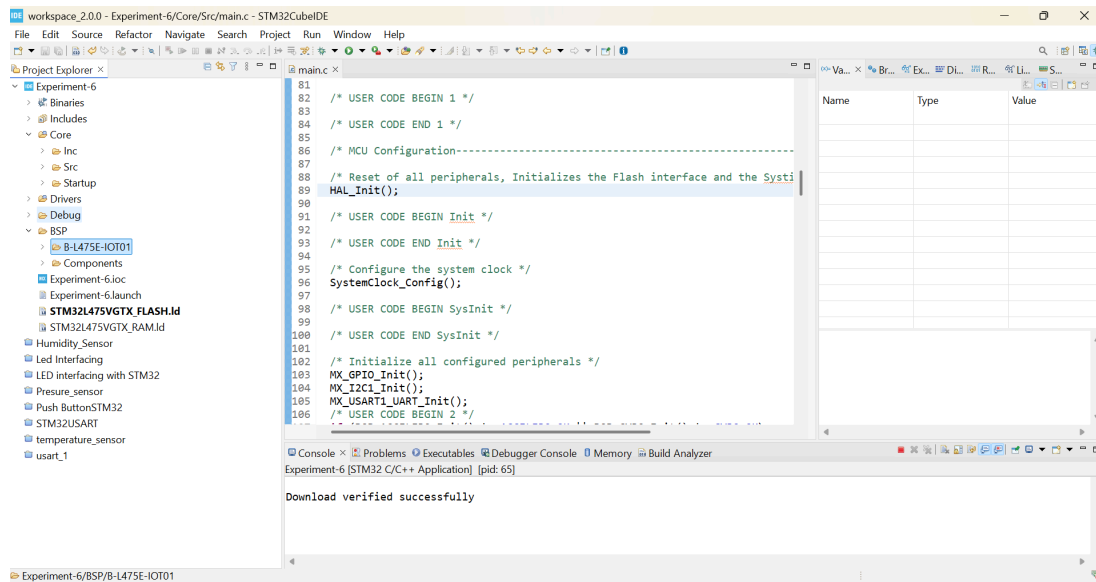


Fig-6.8 – Paste BSP drivers in project folder

17. Delete all folders in BSP, except B-L475E-IOT01 and the component as shown:

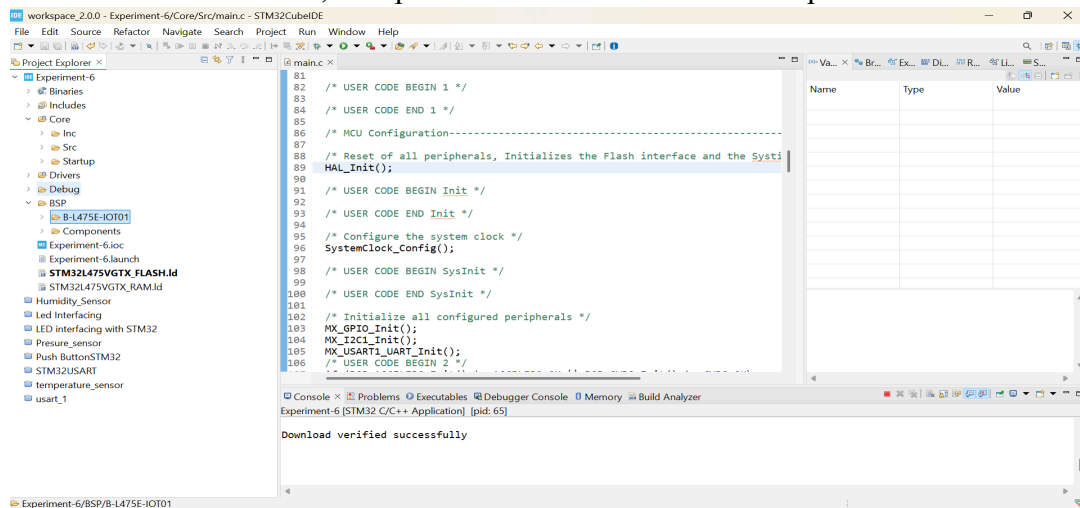


Fig-6.9 – B-L475E-IOT01 and component folder in BSP drivers

18. Keep only these folders in B-L475E-IOT01 and the component as shown in figure:

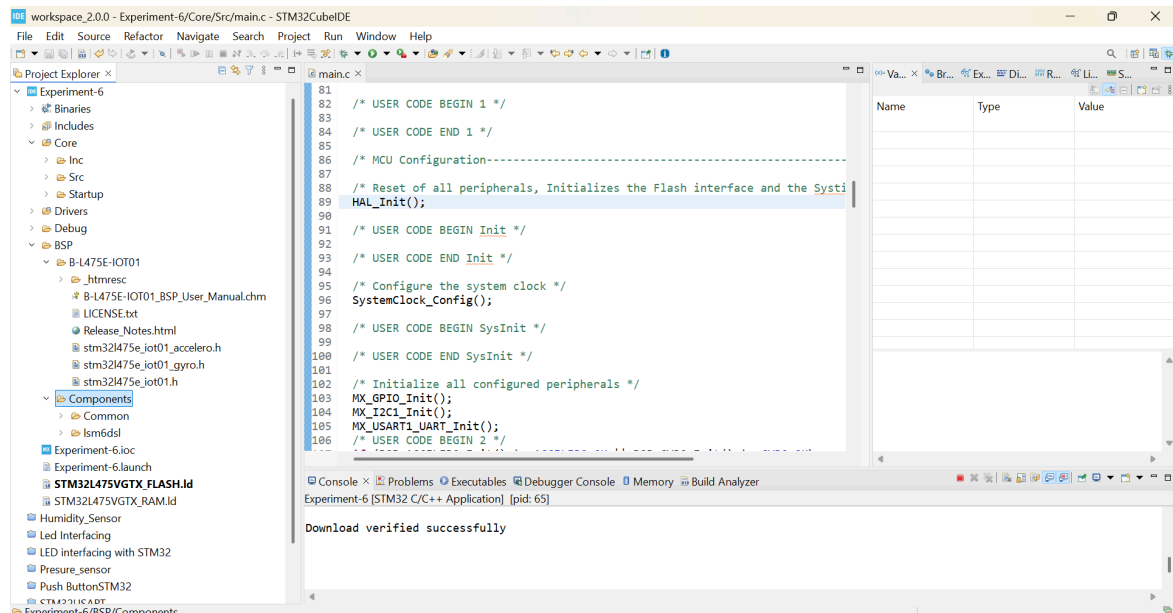


Fig-6.10 – Folders in B-L475E-IOT01 and Component

19. Update include paths: From Project menu or File menu, go to Properties > C/C++ Build > Settings > Tool Settings > MCU GCC Compiler > Include paths and click “add” icon.png to include the new paths as shown below:

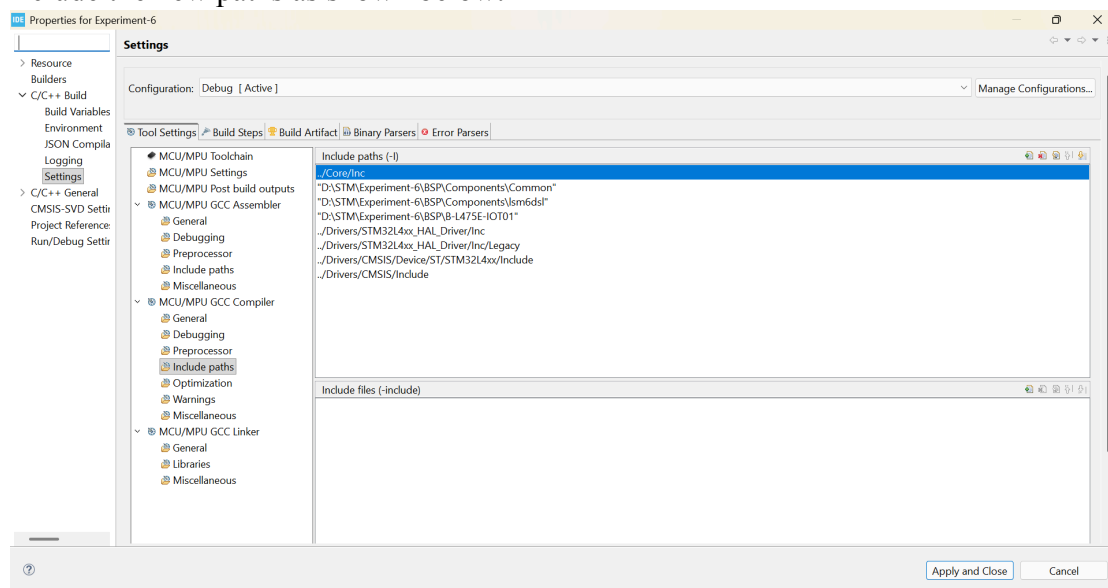


Fig-6.11 – Path settings

20. Add ../Drivers/BSP/B-L475E-IOT01 and ../Drivers/BSP/Components/lsm6dsl paths as addresses given below:
C:\Users\lenovo\STM32Cube\Repository\STM32Cube_FW_L4_V1.17.2\Drivers\BSP\B-L475E-IOT01“and
C:\Users\lenovo\STM32Cube\Repository\STM32Cube_FW_L4_V1.17.2\Drivers\BSP\Components\lsm6dsl

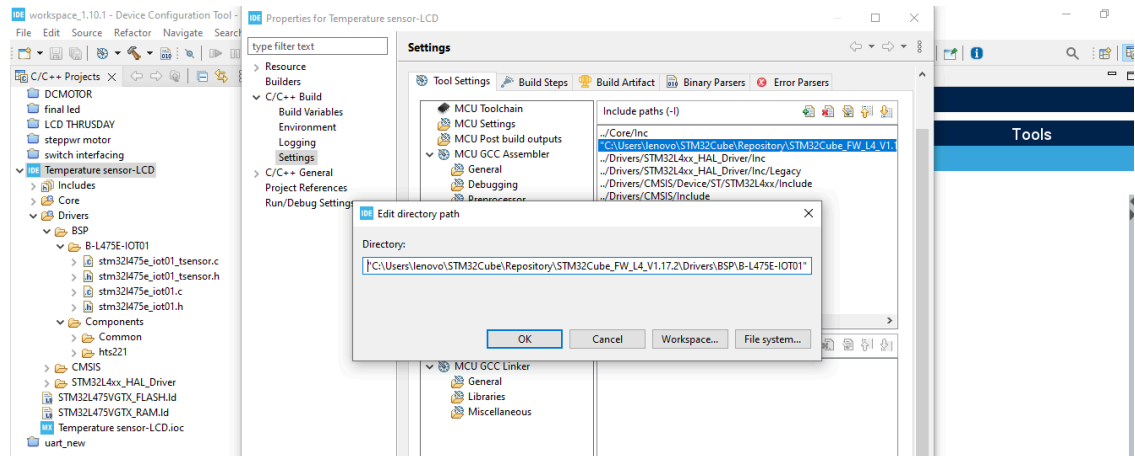


Fig-6.12 – Folders Path settings

21. After adding both paths click “apply and close” as shown below:

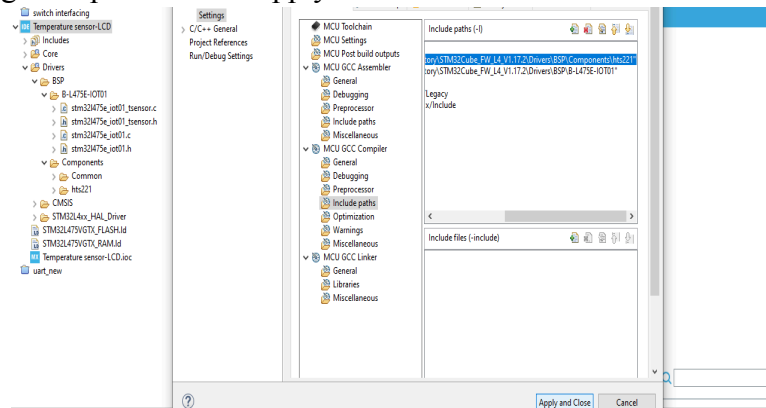


Fig-6.13 – Apply and close setting

22. Go to properties of project>go to settings>go to MCU/MPU settings>Click to float as >apply and close as shown in figure:

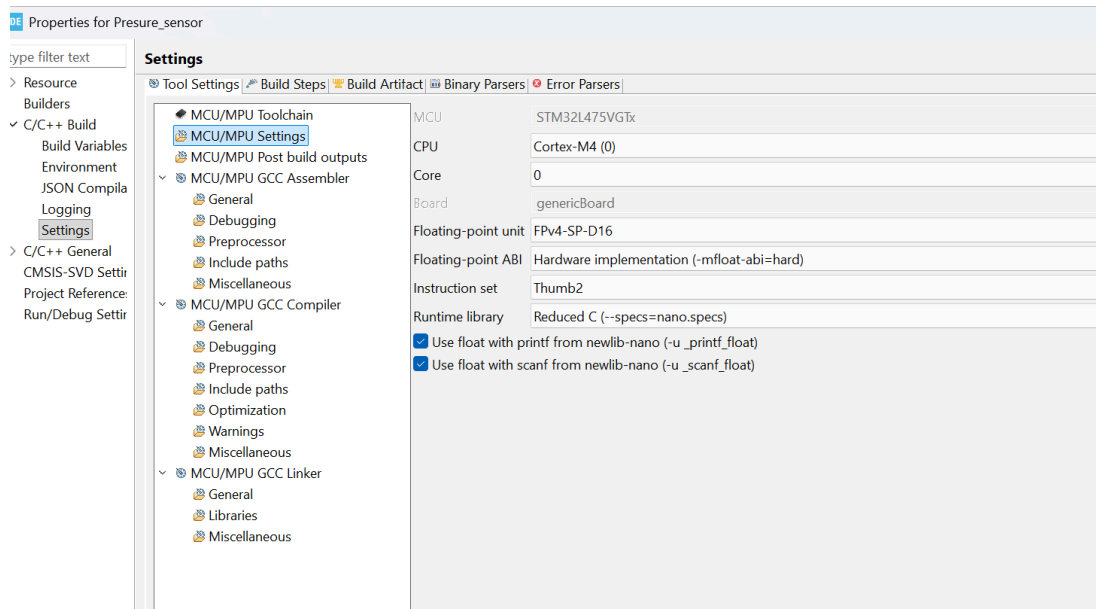


Fig-6.14 – MCU/MPU setting

23. Move lsm6dsl.c, stm32l475e_iot1_accelero.c, stm32l475e_iot1_gyro.c, stm32l475e_iot1.c in src folder from BSP folder.
24. Then “Build” and go to main.c file.
25. Include the header files in main.c files.

```
/* Includes */
#include "stm32l475e_iot01.h"
#include "stm32l475e_iot01_accelero.h"
#include "stm32l475e_iot01_gyro.h"
#include "stdlib.h"
#include <math.h>
#include <string.h>
#include <stdio.h>
```

26. Write the following instructions:

```
/* USER CODE BEGIN PV */
char msg_motion[50] = "LSM6DSL motion sensor initialized\r\n";
char str_accel[100];
char str_gyro[100];
int16_t accel_data[3]; /* Accelerometer usually returns raw 16-bit integers */
float gyro_data[3]; /* FIXED: Gyroscope BSP expects a float array */
/* USER CODE END PV */
```

27. Write the following instructions as given:

```
/* USER CODE BEGIN 2 */
if (BSP_ACCELERO_Init() != ACCELERO_OK || BSP_GYRO_Init() != GYRO_OK)
{
    char errmsg[] = "Motion sensor Init Error\r\n";
```

```

    HAL_UART_Transmit(&huart1, (uint8_t*)errmsg, strlen(errmsg), 1000);
    while(1); // Stop here if init fails
}
else
{
    HAL_UART_Transmit(&huart1, (uint8_t*)msg_motion, strlen(msg_motion), 1000);
}
HAL_Delay(100);
/* USER CODE END 2 */

```

28. Write the following instructions in **while (1) loop** under user code begin 3.

```

BSP_ACCELERO_AccGetXYZ(accel_data);
sprintf(str_accel, "Accelerometer: X=%.2f, Y=%.2f, Z=%.2f mg\r\n",
(float)accel_data[0]/1000.0f, (float)accel_data[1]/1000.0f, (float)accel_data[2]/1000.0f);
HAL_UART_Transmit(&huart1, (uint8_t*)str_accel, strlen(str_accel), 1000);
HAL_Delay(1000);
BSP_GYRO_GetXYZ(gyro_data);
sprintf(str_gyro, "Gyro: X=%.2f, Y=%.2f, Z=%.2f mdps\r\n",
(float)gyro_data[0], (float)gyro_data[1], (float)gyro_data[2]);
HAL_UART_Transmit(&huart1, (uint8_t*)str_gyro, strlen(str_gyro), 1000);
HAL_Delay(1000);

```

29. Now connect the board and click on “Build”, “debug”.

30. Click on “Switch” on popup and click on “Resume”, the program will start execute.

31. Now, open the console “Real term” and set the baud rate as used in program (in this program it is 115200), select port (as per use) and click on “change” on the real term console.

OBSERVATION: The output values accelerometer and gyroscope will display on real term as shown below:

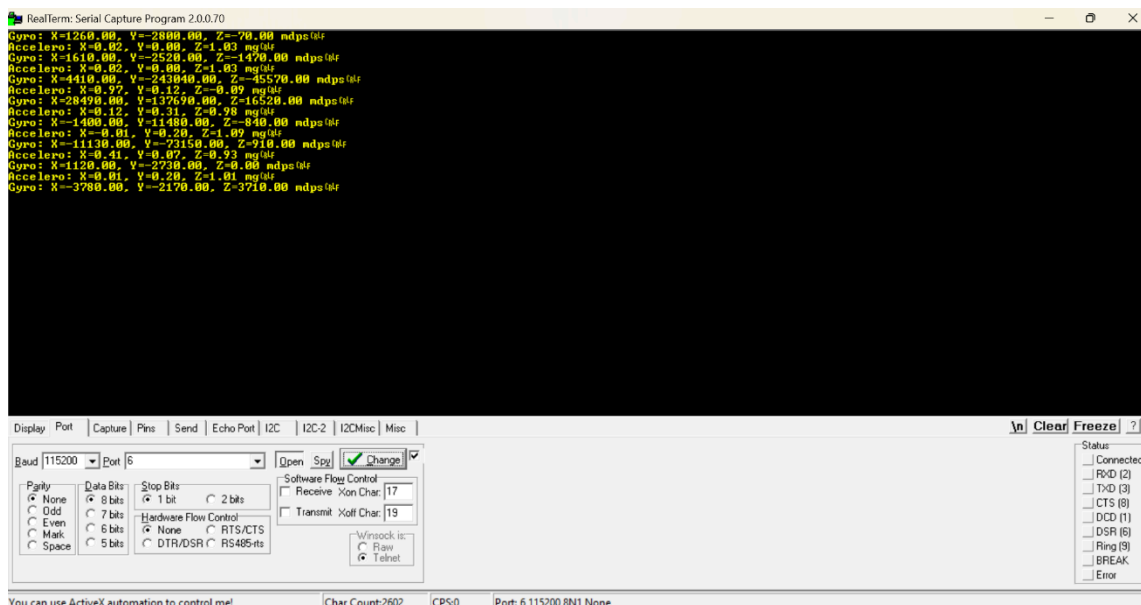


Fig: 6.15 – Output from experiment

Reflection Summary

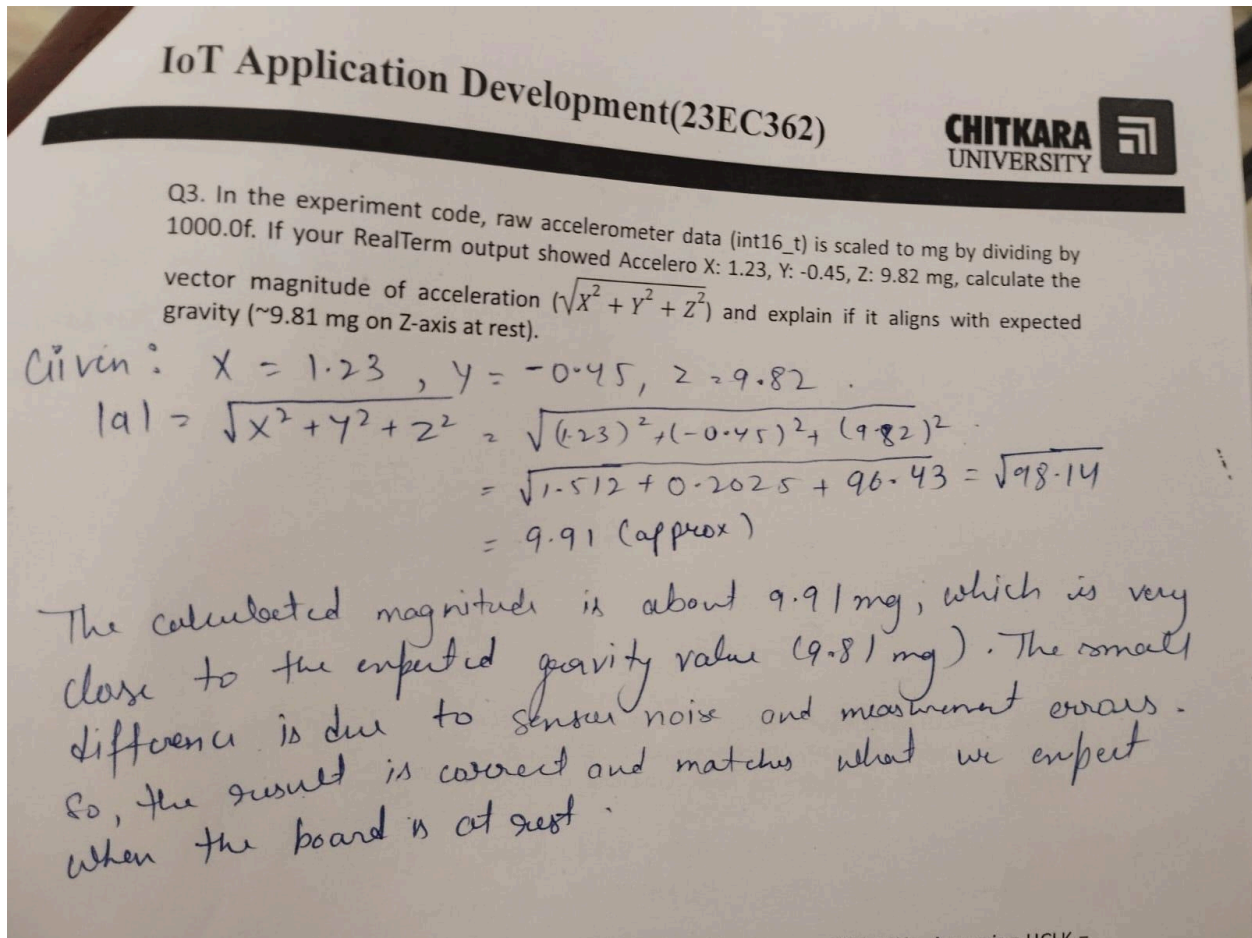
Q1. What challenges did you encounter during BSP driver integration and include path configuration in STM32CubeIDE, and how did resolving them deepen your understanding of board support packages?

While adding the BSP drivers, I faced issues like header files not found and build errors because the include paths were not set properly. Also, some unnecessary folders had to be deleted and only the required ones were kept. After fixing the include paths and placing the required source files in the Src folder, the project compiled correctly. This helped me understand how BSP drivers work and how important correct folder structure and include paths are in STM32CubeIDE.

Q2. How did converting raw 16-bit accelerometer data to mg and gyroscope data to mdps (using sprintf for UART output) illustrate the importance of sensor calibration and data formatting in embedded systems?

The sensor gives raw 16-bit values, which do not directly tell us the real physical values. By converting these raw values into mg for accelerometer and mdps for gyroscope and printing them using sprintf, the data becomes easy to understand. This showed me that sensor data always needs proper scaling and formatting, otherwise the values are not meaningful in real-world units.

Q3. In the experiment code, raw accelerometer data (int16_t) is scaled to mg by dividing by 1000.0f. If your RealTerm output showed Accelero X: 1.23, Y: -0.45, Z: 9.82 mg, calculate the vector magnitude of acceleration ($\sqrt{X^2 + Y^2 + Z^2}$) and explain if it aligns with expected gravity (~9.81 mg on Z-axis at rest).



Q4. The clock was configured for a maximum system frequency of 80 MHz. Assuming HCLK = SYSCLOCK = 80 MHz and the USART1 baud rate was set to 115200, compute the approximate oversampling factor used (typically 8 or 16) and discuss its impact on UART reliability for RealTerm data streaming.

In this experiment, the system clock (HCLK = SYSCLOCK) is configured to 80 MHz and the USART1 baud rate is set to 115200. In STM32 microcontrollers, the USART typically uses an oversampling factor of 16 (sometimes 8, but 16 is the default and more common).

Oversampling means that instead of sampling each bit only once, the USART samples each bit 16 times and then decides whether the bit is 0 or 1 based on these samples. This helps in getting more accurate data, especially when there is noise on the line or small timing errors between the transmitter and receiver.

With an 80 MHz clock, the USART has enough clock cycles to generate an accurate baud rate of 115200 using oversampling by 16. This results in:

- Better timing accuracy of each bit,
- Less chance of bit errors,
- More stable and reliable serial communication.

Because of this, the data sent from the STM32 to RealTerm is received correctly without missing or corrupted characters. So, using oversampling (especially 16x) improves the reliability and quality of UART data streaming in this experiment.

Q5. Reflecting on the clock configuration (to 80 MHz) and peripheral setup (USART1 async on PB6/PB7, I2C1_SCL on PB8) in this LSM6DSL experiment, what key principles of STM32 pin multiplexing and peripheral resource allocation will you apply to future multi-sensor embedded projects?

In this experiment, we used PB6 and PB7 for USART1 and PB8 for I2C1, which shows that STM32 pins have multiple alternate functions and must be assigned carefully. Only one function can be used on a pin at a time, so proper pin planning is important to avoid conflicts. Also, only the required peripherals should be enabled to keep the system simple and reliable. The 80 MHz clock configuration ensures that all peripherals work properly without communication errors. This experiment helped me understand how to plan pins, allocate peripherals, and configure clocks for multi-sensor embedded projects.